INSTITUTE FOR ADVANCED COMPUTING

AND

SOFTWARE DEVELOPMENT,

AKURDI, PUNE

DOCUMENTATION ON

"End-to-End DevSecOps and GitOps Kubernetes Platform with SonarQube, Trivy, Automated CI/CD, Monitoring, & Canary Deployments"

PG-DITISS August 2025

SUBMITTED BY

GROUP NO: - 07

SUMIT SANJAY BADGUJAR (258439)

KETAN GYANESHWAR MAHAJAN (258451)

MRS. SUSHMA HATTARKI
PROJECT GUIDE

MR. ANIL SHARMA
CENTRE CO-ORDINATOR

ABSTRACT

With the increasing adoption of cloud-native applications, ensuring secure and reliable software delivery has become a major challenge for organizations. Traditional DevOps pipelines focus mainly on automation and speed, often treating security as a later-stage concern, which increases the risk of vulnerabilities being deployed into production environments.

This project presents a **Secure GitOps-Based DevSecOps Platform** that integrates security throughout the entire CI/CD lifecycle. The platform automates code analysis, dependency scanning, container security, deployment, and monitoring using tools such as **Jenkins, SonarQube, Docker, Kubernetes, Argo CD, Trivy, and OWASP Dependency-Check**. Infrastructure is provisioned on **AWS using Terraform**, ensuring consistency through Infrastructure as Code.

A GitOps-based deployment approach is implemented using Argo CD, enabling declarative, version-controlled deployments with improved traceability and rollback capability. Continuous monitoring and alerting are achieved using **Prometheus, Grafana, and Alertmanager**. The proposed system demonstrates how DevSecOps and GitOps practices can be combined to deliver applications that are secure, scalable, and production-ready.

TABLE OF CONTENTS

LIST OF ABBREVIATIONS

Sr. No.	Abbreviation	Full Form
1	CI/CD	Continuous Integration and Continuous Deployment
2	DevOps	Development and Operations
3	DevSecOps	Development, Security, and Operations
4	GitOps	Git-based Operations
5	IaC	Infrastructure as Code
6	AWS	Amazon Web Services
7	Docker	Docker Container Platform
8	Kubernetes (K8s)	Container Orchestration Platform
9	SAST	Static Application Security Testing
10	OWASP	Open Web Application Security Project
11	CVE	Common Vulnerabilities and Exposures
12	RBAC	Role-Based Access Control

1. INTRODUCTION

The rapid growth of cloud computing and microservices-based architectures has significantly changed the way modern software applications are developed and deployed. Organizations are required to deliver applications quickly while maintaining reliability, scalability, and security. DevOps practices have emerged to automate software development and deployment processes, reducing manual effort and improving delivery speed.

However, traditional DevOps pipelines primarily focus on automation and operational efficiency, often treating security as a separate or post-deployment concern. This approach leads to vulnerabilities being identified late in the development lifecycle, increasing remediation costs and exposing production systems to potential security risks. With the rise in software supply chain attacks and container-based vulnerabilities, integrating security into the CI/CD pipeline has become essential.

DevSecOps extends DevOps by embedding security controls at every stage of the development process. Alongside this, GitOps introduces a declarative operational model where Git repositories act as the single source of truth for application and infrastructure configurations. GitOps enables consistent deployments, version control, and easy rollback mechanisms.

This project focuses on developing a **Secure GitOps-Based DevSecOps Platform** that integrates CI/CD automation, security scanning, containerization, and orchestration using tools such as **Jenkins, SonarQube, Docker, Kubernetes, Argo CD, and AWS**. The proposed platform demonstrates how DevSecOps and GitOps practices can be combined to deliver secure, reliable, and scalable cloud-native applications.

1.1. PROBLEM STATEMENT

Modern software delivery requires applications to be developed and deployed rapidly while maintaining strong security and operational reliability. Although DevOps practices have improved automation and reduced deployment time, security is often treated as a separate or post-deployment activity. This results in vulnerabilities within source code, third-party dependencies, and container images being detected late in the development lifecycle, increasing the risk of security breaches in production environments.

Traditional CI/CD pipelines also lack standardized mechanisms for infrastructure automation, version-controlled deployments, and effective rollback strategies. Manual configuration and deployment processes can lead to inconsistencies across environments and make system maintenance difficult. Furthermore, limited monitoring and alerting capabilities reduce visibility into application performance and infrastructure health, delaying issue detection and resolution.

There is a need for an integrated platform that combines CI/CD automation, continuous security enforcement, infrastructure as code, and real-time monitoring. Such a platform should prevent insecure artifacts from reaching production, ensure consistent and auditable deployments, and provide continuous visibility into system operations. This project addresses these challenges by implementing a **Secure GitOps-Based DevSecOps Platform** that unifies automation, security, and operational best practices into a single workflow.

2. LITERATURE SURVEY

The field of software delivery has evolved significantly with the adoption of DevOps practices, which aim to improve collaboration between development and operations teams while automating build, test, and deployment processes. Tools such as Jenkins have been widely used to implement CI/CD pipelines, enabling faster and more reliable software releases. However, early DevOps implementations primarily focused on speed and automation, with limited emphasis on security integration.

To address security challenges, DevSecOps emerged as an extension of DevOps, promoting the integration of security practices throughout the software development lifecycle. Static application security testing (SAST) tools like SonarQube are commonly used to identify code quality issues and security vulnerabilities during the early stages of development. Additionally, software composition analysis (SCA) tools and container security scanners such as Trivy help detect vulnerabilities in third-party dependencies and container images. While these tools are effective individually, they often require manual integration and coordination within CI/CD pipelines.

Containerization using Docker has become a standard approach for packaging applications, providing consistency across development and production environments. Kubernetes further enhances this by offering container orchestration features such as scalability, self-healing, and automated deployments. Despite these advantages, managing Kubernetes deployments manually can be complex and error-prone, especially in large-scale environments.

GitOps has been introduced as a modern operational model to simplify Kubernetes application management. Tools like Argo CD use Git repositories as the single source of truth for application and infrastructure configurations, enabling declarative deployments, version control, and automated rollback. This approach improves deployment consistency and auditability compared to traditional deployment methods.

Monitoring and observability tools such as Prometheus and Grafana are widely used to collect metrics and visualize system performance, while Alertmanager provides automated alerting for system failures. Although these tools offer strong visibility, their effectiveness depends on proper integration with CI/CD and deployment workflows.

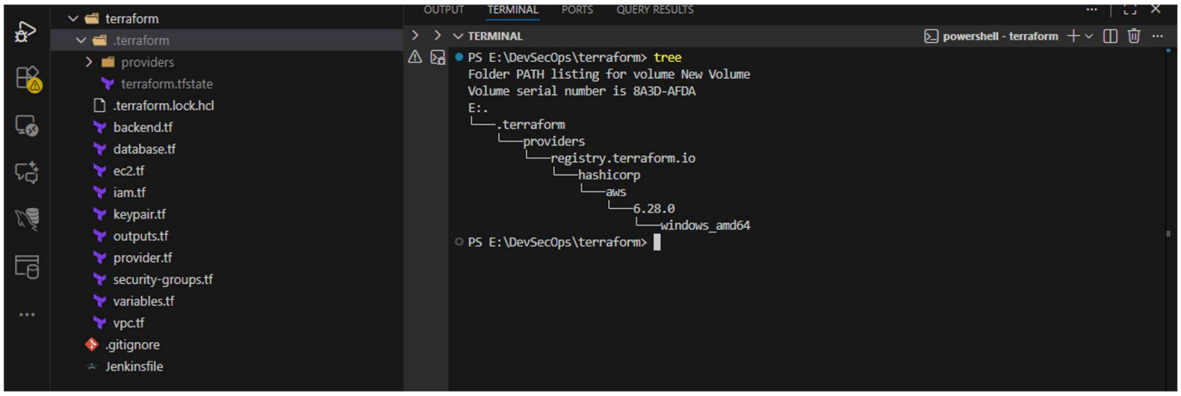
Existing studies and implementations highlight the benefits of DevOps, DevSecOps, and GitOps practices independently. However, there is a need for an integrated platform that combines CI/CD automation, continuous security enforcement, GitOps-based deployment, and real-time monitoring into a unified workflow. This project builds upon existing tools and research by integrating them into a comprehensive **Secure GitOps-Based DevSecOps Platform**, addressing the limitations identified in previous approaches.

3. METHODOLOGY

The methodology of this project follows a systematic, step-by-step approach to implement a secure, automated, and scalable DevSecOps platform using GitOps principles. Each step represents a distinct phase in the development and deployment lifecycle.

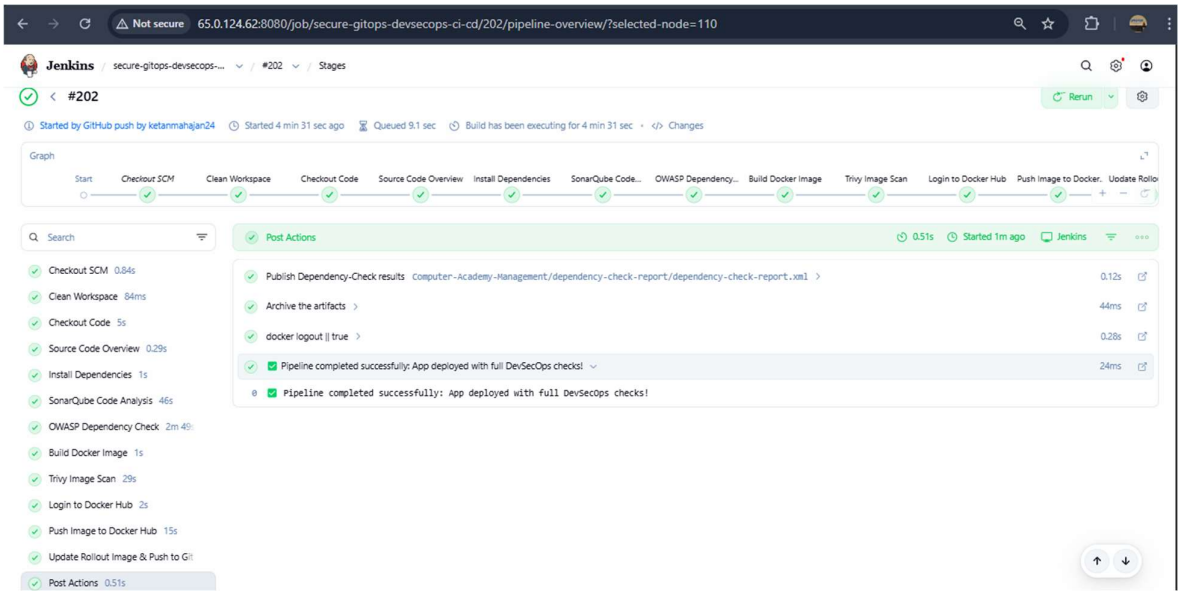
Step 1: Infrastructure Provisioning

The first step involves provisioning the required cloud infrastructure using **Infrastructure as Code (IaC)**. Terraform is used to create and manage AWS resources such as Virtual Private Cloud (VPC), EC2 instances, security groups, and networking components. This approach ensures consistent, repeatable, and version-controlled infrastructure setup.



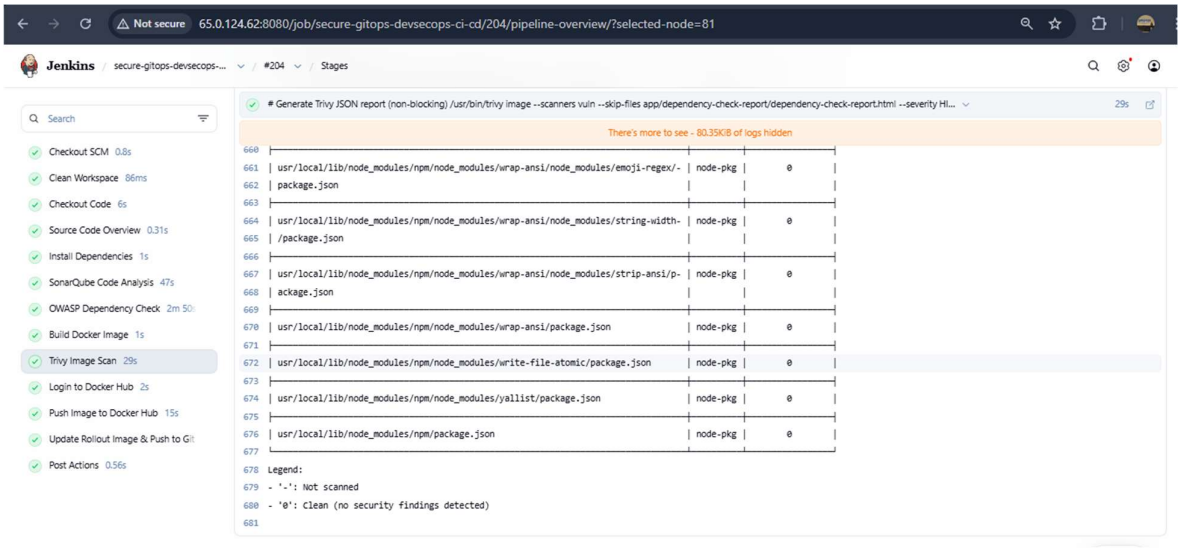
Step 2: Version Control and Source Code Management

All application source code, configuration files, and Kubernetes manifests are stored in a Git repository. Git acts as the single source of truth for both application and infrastructure changes. Any modification to the code or configuration is tracked and versioned, enabling collaboration and traceability.



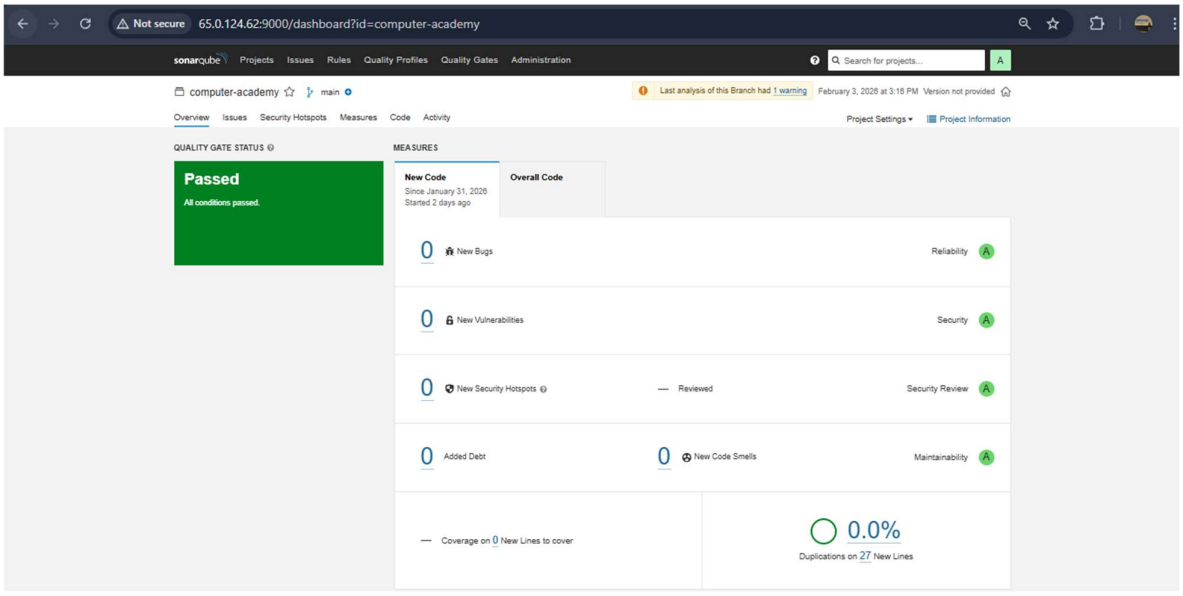
Step 3: CI/CD Pipeline Setup

A Continuous Integration and Continuous Deployment (CI/CD) pipeline is configured using **Jenkins**. The pipeline is automatically triggered whenever code is pushed to the Git repository. This stage automates the build and initial testing of the application.



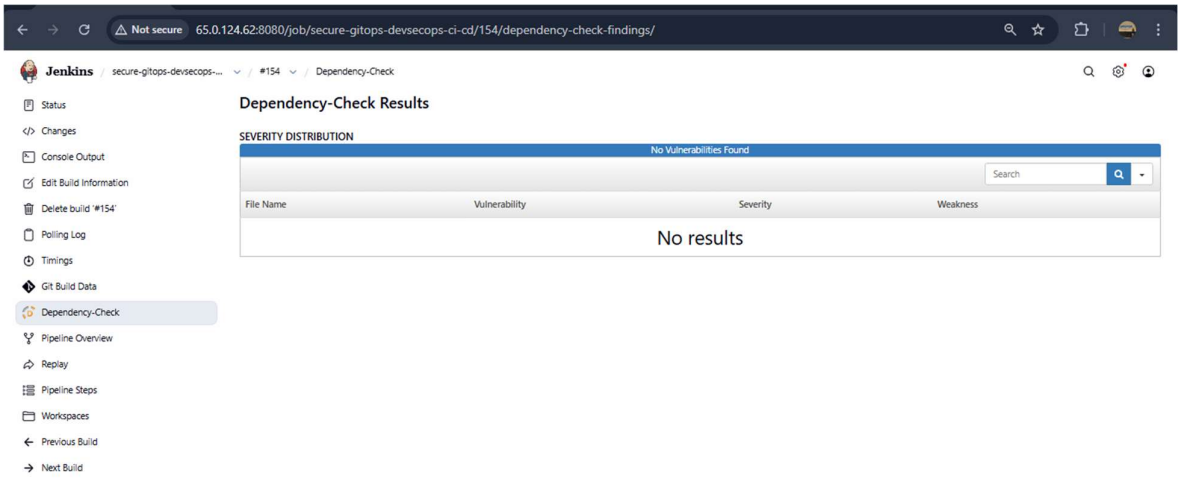
Step 4: Static Code Analysis

During the CI phase, **SonarQube** is integrated to perform static code analysis. It evaluates code quality, detects bugs, code smells, and potential security vulnerabilities. If the code does not meet defined quality gates, the pipeline fails, preventing insecure code from progressing further.



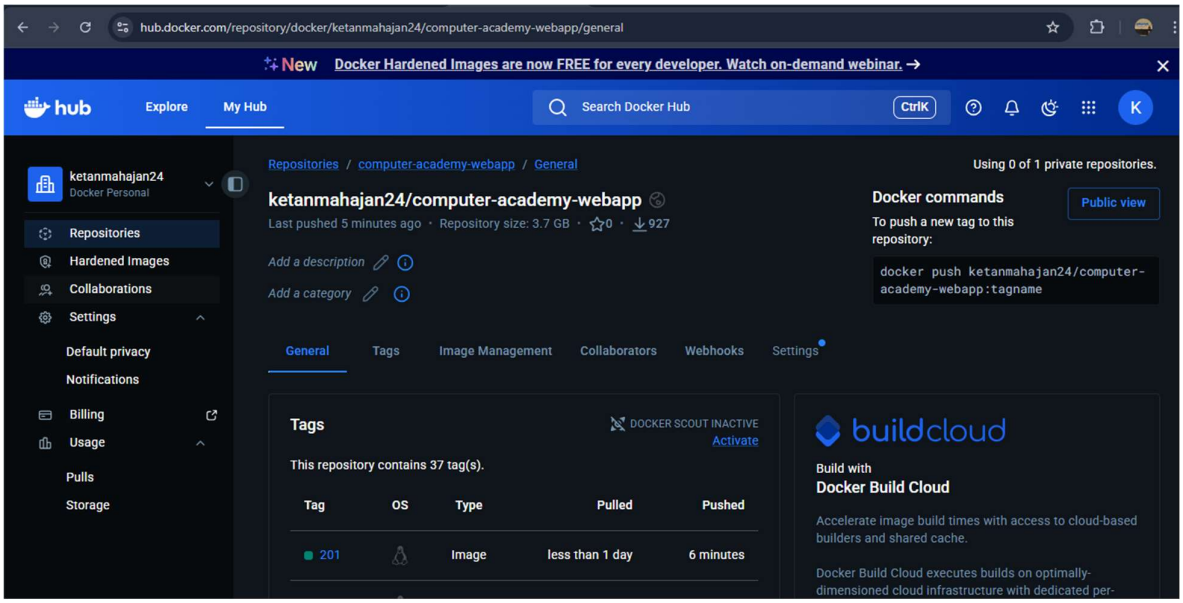
Step 5: Dependency and Container Security Scanning

Security is enhanced by scanning third-party dependencies and container images. **OWASP Dependency-Check** is used to identify known vulnerabilities in libraries, while **Trivy** scans Docker images for security issues. Any critical vulnerabilities detected cause the pipeline to stop automatically.



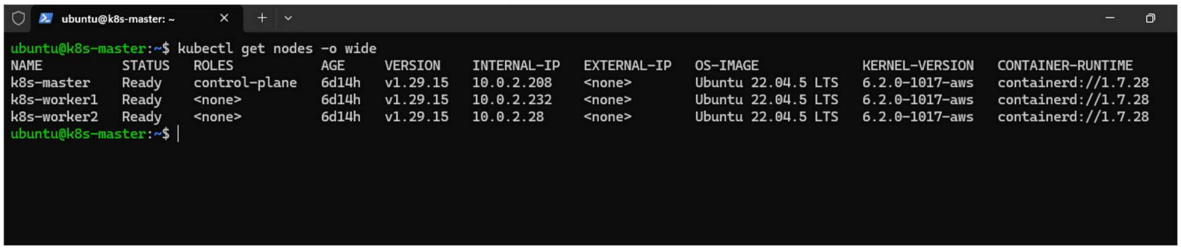
Step 6: Application Containerization

Once the code passes all security checks, the application is containerized using **Docker**. A Docker image is built and pushed to a container registry. Containerization ensures consistency across development, testing, and production environments.



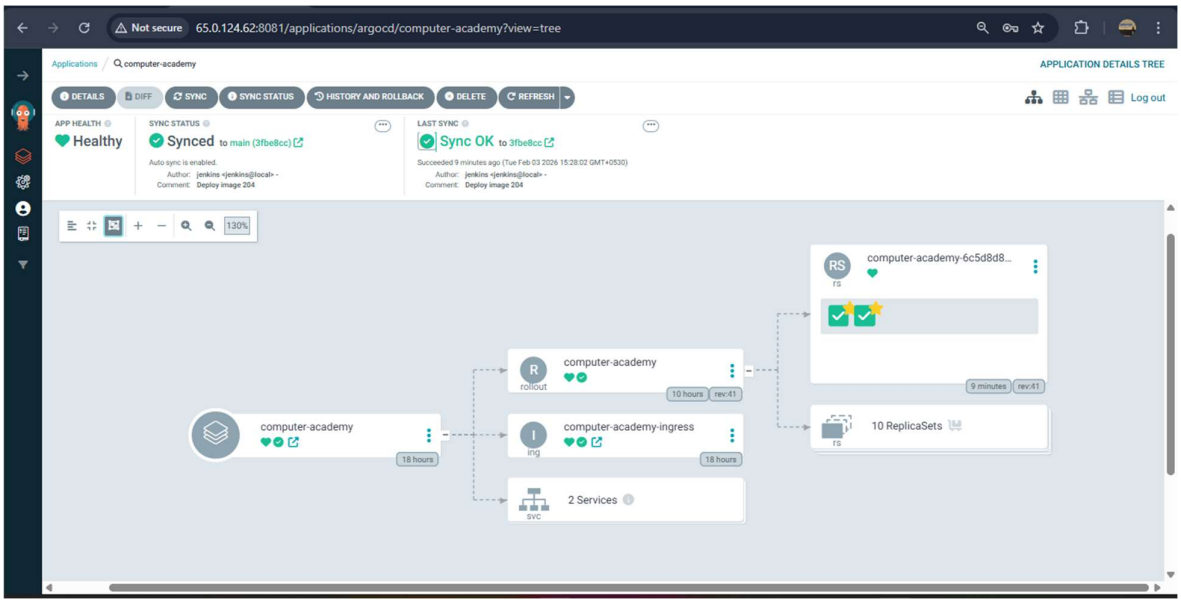
Step 7: Kubernetes Cluster Deployment

The containerized application is deployed on a **Kubernetes cluster**, which provides orchestration, scalability, and self-healing capabilities. Kubernetes manages container scheduling, service discovery, and resource utilization.



Step 8: GitOps-Based Deployment

A GitOps approach is implemented using **Argo CD**. Kubernetes deployment manifests are stored in a Git repository, and Argo CD continuously monitors the repository to synchronize the desired state with the cluster. This enables automated deployments, version-controlled changes, and easy rollback in case of failures.

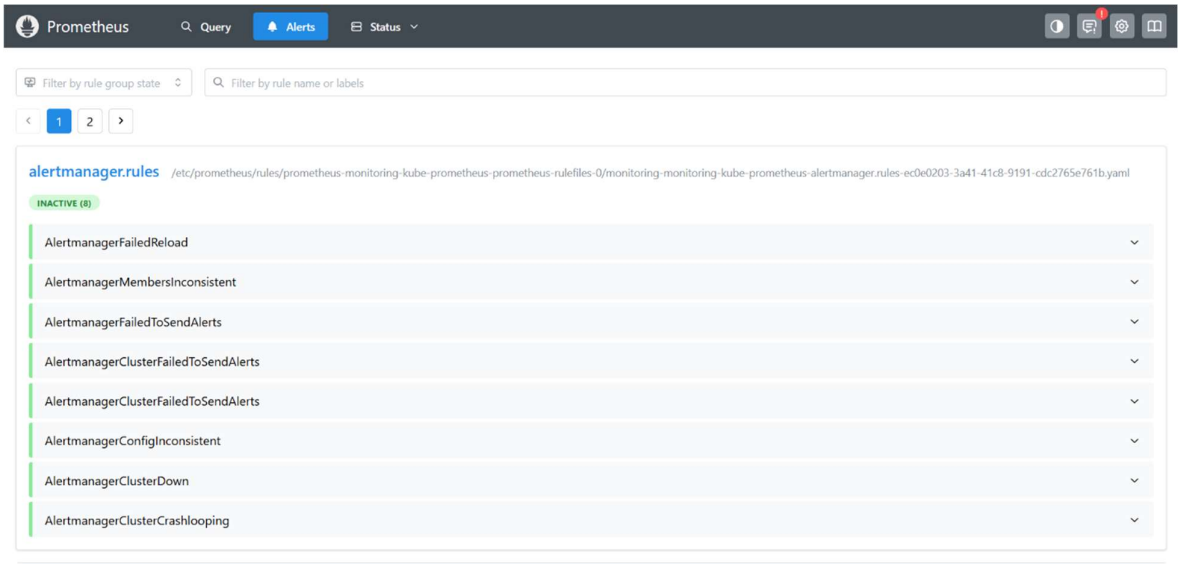


Step 9: Monitoring and Alerting

To ensure operational visibility, **Prometheus** is used to collect metrics from the application and infrastructure. **Grafana** visualizes these metrics through dashboards, while **Alertmanager** sends alerts when predefined thresholds are exceeded, enabling proactive issue resolution.

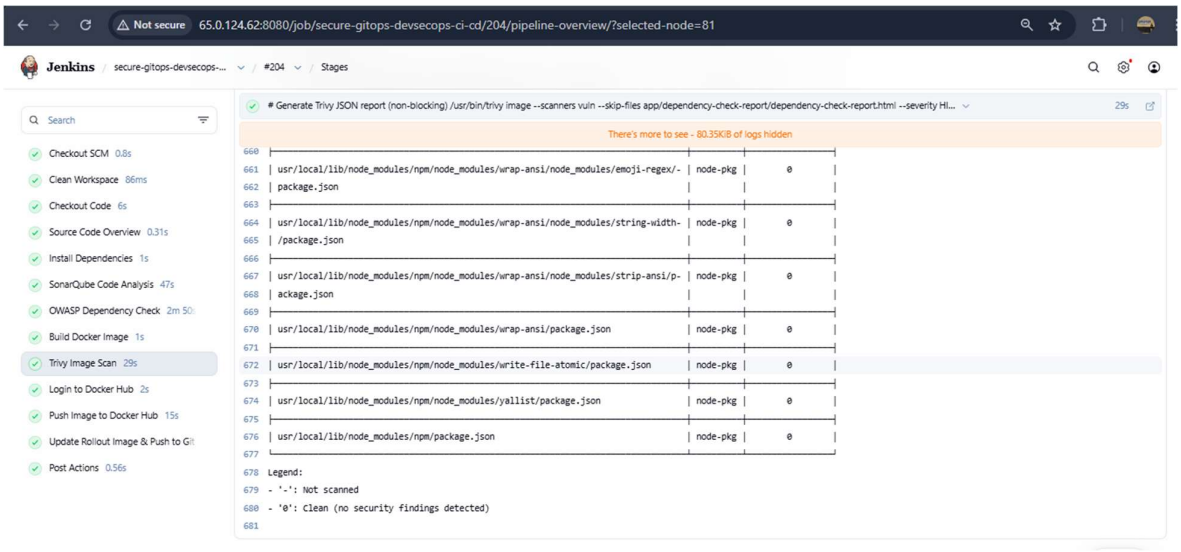
(Grafana) Monitoring:

(Prometheus) Alerting:



Step 10: Validation and Testing

The final step involves validating the complete pipeline through functional testing, security testing, and monitoring verification. This ensures that the system meets the objectives of secure deployment, continuous monitoring, and reliable application delivery.



- **Comparison of DevOps and DevSecOps Practices:**

Aspect	DevOps	DevSecOps
Security Integration	Implemented after deployment	Integrated throughout CI/CD pipeline
Focus	Speed and automation	Speed, automation, and security
Vulnerability Detection	Late-stage	Early (Shift-Left Security)

3.1.SYSTEM ARCHITECTURE

The system architecture of the **Secure GitOps-Based DevSecOps Platform** is designed to automate the complete software delivery lifecycle while embedding security and operational best practices at every stage. The architecture follows a layered approach consisting of source control, CI/CD automation, security enforcement, container orchestration, GitOps deployment, and monitoring.

Architecture Diagram Explanation

1. Developers commit code to the Git repository.
2. Jenkins is triggered automatically and executes the CI pipeline.
3. SonarQube performs static code analysis.
4. OWASP Dependency-Check and Trivy enforce security scanning.
5. Docker builds the container image and pushes it to the registry.
6. Argo CD pulls Kubernetes manifests from Git and deploys them to the cluster.
7. Kubernetes manages application runtime and scaling.
8. Prometheus, Grafana, and Alertmanager provide monitoring and alerting.

Architecture Diagram:

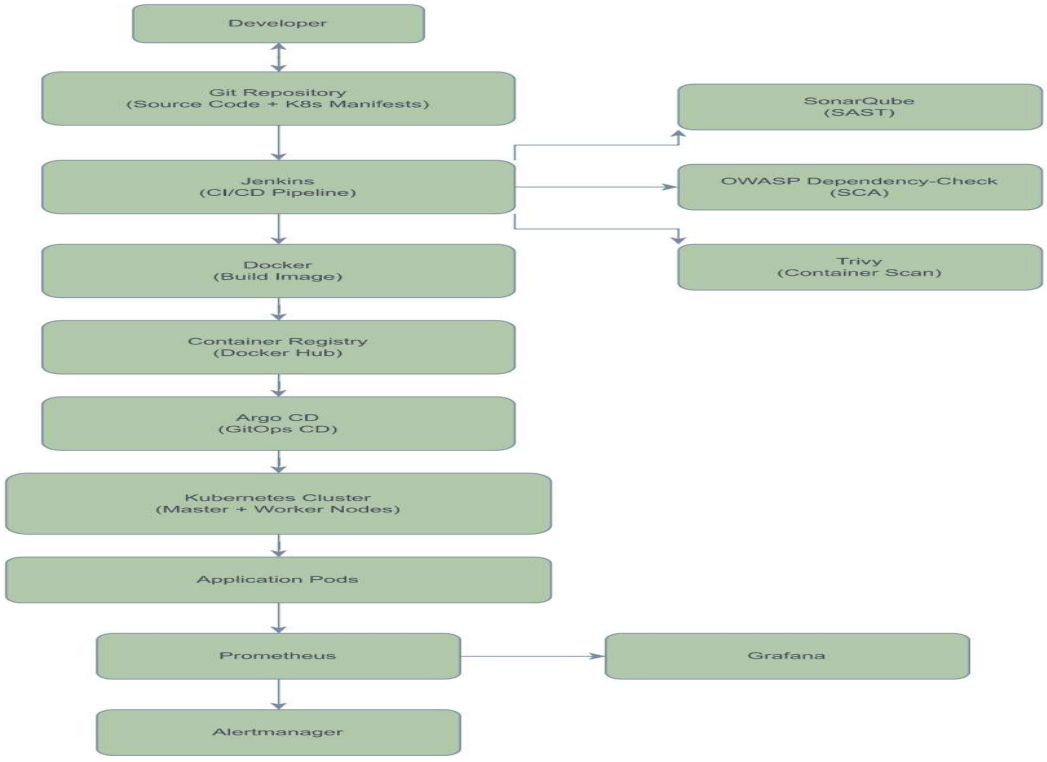


Figure 1 illustrates the overall DevSecOps and GitOps architecture of the system.

4. REQUIREMENT SPECIFICATION

This section describes the software and hardware requirements necessary for implementing the Secure GitOps-Based DevSecOps Platform. The requirements are defined to ensure smooth automation, security integration, and reliable deployment of applications.

4.1 SOFTWARE REQUIREMENTS

The following software components are required for the successful implementation of the project:

- **Operating System:** Ubuntu 22.04 LTS
- **Version Control System:** Git and GitHub
- **CI/CD Tool:** Jenkins
- **Static Code Analysis Tool:** SonarQube
- **Security Tools:** OWASP Dependency-Check, Trivy
- **Containerization Platform:** Docker
- **Container Orchestration:** Kubernetes
- **GitOps Tool:** Argo CD
- **Monitoring Tools:** Prometheus, Grafana, Alertmanager
- **Infrastructure as Code Tool:** Terraform
- **Cloud Platform:** Amazon Web Services (AWS)
- **Scripting Languages:** Bash, YAML

4.2 HARDWARE REQUIREMENTS

The following hardware resources are required to support the DevSecOps platform:

- **Processor:** Minimum 2 vCPU (recommended 4 vCPU)
- **Memory (RAM):** Minimum 4 GB (recommended 8 GB or higher)
- **Storage:** Minimum 50 GB SSD
- **Network:** Stable internet connectivity
- **Kubernetes Cluster:**
    - 1 Master Node
    - 2 Worker Nodes

- **Tools and Technologies Used in the Project:**

Category	Tool / Technology	Purpose
Version Control	Git, GitHub	Source code & configuration management
CI/CD	Jenkins	Pipeline automation
Code Quality	SonarQube	Static code analysis
Security	OWASP Dependency-Check, Trivy	Vulnerability scanning
Containerization	Docker	Application packaging
Orchestration	Kubernetes	Container orchestration
GitOps	Argo CD	Declarative deployments
Monitoring	Prometheus, Grafana	Metrics & visualization
Alerting	Alertmanager	Failure notifications
IaC	Terraform	Infrastructure provisioning
Cloud	AWS	Hosting infrastructure

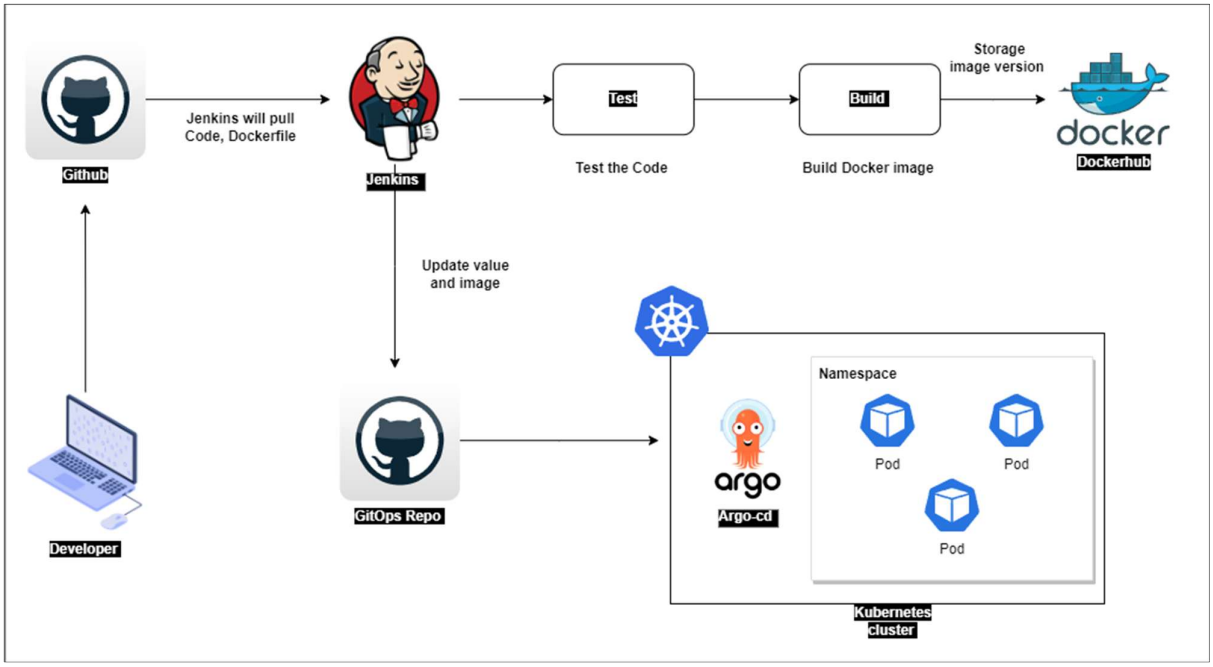


Fig: CI/CD Pipeline Workflow using Jenkins

5. WORKING

The working of the **Secure GitOps-Based DevSecOps Platform** demonstrates how automation, security, and deployment are integrated into a single continuous workflow. The system operates through a sequence of well-defined stages, starting from source code commit to application deployment and monitoring.

Phase 1: Source Code Commit

The working process begins when a developer commits application source code or configuration changes to the Git repository. The repository acts as the single source of truth for application code, Docker configurations, and Kubernetes manifests.

Phase 2: Continuous Integration Trigger

Once the code is pushed to the repository, a webhook automatically triggers the Jenkins CI/CD pipeline. Jenkins retrieves the latest code and initiates the build process without manual intervention.

Phase 3: Static Code Analysis

During the CI phase, Jenkins integrates with SonarQube to perform static code analysis. SonarQube evaluates code quality, identifies bugs, code smells, and detects potential security vulnerabilities. If the code fails to meet predefined quality gates, the pipeline is terminated.

Phase 4: Dependency and Container Security Scanning

After successful code analysis, the pipeline performs security scans on third-party dependencies using OWASP Dependency-Check. Docker images are then scanned using Trivy to identify known vulnerabilities. Any critical or high-severity vulnerabilities cause the pipeline to fail, ensuring only secure artifacts proceed further.

Phase 5: Application Containerization

Once security validations are completed, the application is containerized using Docker. A Docker image is built and pushed to a container registry, making it available for deployment.

Phase 6: GitOps-Based Deployment

Kubernetes deployment manifests stored in the Git repository define the desired application state. Argo CD continuously monitors these manifests and automatically deploys or updates the application in the Kubernetes cluster to match the declared configuration.

Phase 7: Container Orchestration

The Kubernetes cluster manages application pods, services, scaling, and self-healing. In case of failures, Kubernetes automatically restarts or reschedules pods to maintain application availability.

Phase 8: Monitoring and Alerting

Prometheus collects real-time metrics from the application and Kubernetes cluster. Grafana visualizes these metrics through dashboards, while Alertmanager sends alerts in case of abnormal conditions, enabling proactive issue resolution.

6. IMPLEMENTATION

The implementation of the **Secure GitOps-Based DevSecOps Platform** focuses on integrating automation, security, deployment, and monitoring into a unified CI/CD workflow. The system is implemented using open-source and cloud-native tools, following industry best practices.

6.1 AWS Infrastructure using Terraform

The AWS infrastructure for the DevSecOps platform is provisioned using **Terraform**, an Infrastructure as Code (IaC) tool. Terraform enables automated, consistent, and repeatable creation of cloud resources while maintaining version control over infrastructure configurations.

The infrastructure includes a Virtual Private Cloud (VPC) with public and private subnets, EC2 instances for the CI/CD server, Kubernetes master and worker nodes, and necessary security groups. Terraform scripts define networking components, instance specifications, and access rules. By using Terraform, manual configuration errors are minimized, and infrastructure scalability and reproducibility are ensured.

6.2 CI/CD Pipeline using Jenkins

A Continuous Integration and Continuous Deployment (CI/CD) pipeline is implemented using **Jenkins**. Jenkins is configured to automatically trigger pipeline execution whenever changes are pushed to the Git repository through webhooks. The pipeline automates source code checkout, build execution, security analysis, and deployment preparation.

Pipeline-as-Code is implemented using a Jenkinsfile, which defines all pipeline stages in a declarative manner. This ensures consistency, traceability, and easy modification of the CI/CD workflow.

6.3 Static Code Analysis using SonarQube

To maintain code quality and enforce security standards, **SonarQube** is integrated into the Jenkins pipeline. During the CI phase, SonarQube analyzes the source code to detect bugs, code smells, and security vulnerabilities.

Quality gates are configured to define acceptable thresholds for code coverage and security issues. If the application fails to meet these standards, the pipeline is terminated automatically, preventing insecure or low-quality code from progressing further.

6.4 Dependency & Container Security (OWASP Dependency-Check, Trivy)

Security scanning is extended beyond source code analysis by integrating **OWASP Dependency-Check** and **Trivy**. OWASP Dependency-Check identifies known vulnerabilities in third-party libraries by referencing public vulnerability databases.

Trivy is used to scan Docker images for vulnerabilities in operating system packages and application dependencies. If critical vulnerabilities are detected, the pipeline fails automatically, enforcing a shift-left security approach and ensuring only secure artifacts are deployed.

6.5 Docker Containerization

After passing all security checks, the application is containerized using **Docker**. A Dockerfile defines the application environment, dependencies, and runtime configuration. Docker images are built during the CI process and pushed to a container registry.

Containerization ensures consistency across development, testing, and production environments and simplifies application deployment and scalability.

6.6 Kubernetes Cluster Setup & Deployment

The containerized application is deployed on a **Kubernetes cluster** consisting of a master node and multiple worker nodes. Kubernetes manages container orchestration, service discovery, scaling, and self-healing of application pods.

Deployment and service configurations are defined using YAML manifests. Kubernetes automatically schedules pods, monitors their health, and restarts them in case of failures, ensuring high availability.

6.7 GitOps Deployment using Argo CD

A GitOps-based deployment model is implemented using **Argo CD**. Kubernetes manifests stored in a Git repository define the desired state of the application. Argo CD continuously monitors the repository and synchronizes changes with the Kubernetes cluster.

This approach ensures declarative, version-controlled, and auditable deployments. Rollback to previous stable versions can be performed easily by reverting changes in the Git repository.

6.8 Monitoring & Alerting (Prometheus, Grafana, Alertmanager)

Monitoring and observability are implemented using **Prometheus**, **Grafana**, and **Alertmanager**. Prometheus collects real-time metrics from the Kubernetes cluster and deployed applications. Grafana visualizes these metrics through interactive dashboards.

Alertmanager is configured to send alerts when predefined thresholds are exceeded, enabling proactive detection and resolution of system issues. This monitoring stack ensures continuous visibility into system performance, reliability, and health.

- **AWS Infrastructure Components:**

Component	Description
VPC	Isolated network for cloud resources
EC2	Compute instances for CI/CD & Kubernetes
Public Subnet	Jenkins & Bastion access
Private Subnet	Kubernetes nodes & database
Security Groups	Network access control
IAM	Access and permission management
S3	Terraform state storage
NAT Gateway	Internet access for private nodes

7. APPLICATIONS

The Secure GitOps-Based DevSecOps Platform developed in this project has wide applicability in modern software development and cloud environments. By integrating automation, security, infrastructure as code, and continuous monitoring, the platform addresses real-world challenges faced by organizations adopting cloud-native technologies.

1. Enables enterprise-level DevSecOps adoption with automated and secure CI/CD pipelines.
2. Provides a secure CI/CD framework for cloud-native applications.
3. Supports Kubernetes-based deployment with scalability and high availability.
4. Implements GitOps-based deployments using Argo CD for consistency and rollback.
5. Automates cloud infrastructure provisioning using Terraform.
6. Enhances software supply chain security through continuous vulnerability scanning.
7. Provides real-time monitoring and alerting using Prometheus and Grafana.
8. Supports fast incident response and recovery through Git-based rollback.
9. Manages multiple environments such as development, staging, and production.
10. Serves as a learning and proof-of-concept platform for DevSecOps and cloud automation.

- **CI/CD Pipeline Stages:**

Stage No.	Pipeline Stage	Description
1	Code Commit	Developer pushes code to Git
2	CI Trigger	Jenkins pipeline starts
3	Code Analysis	SonarQube scans source code
4	Dependency Scan	OWASP Dependency-Check
5	Container Scan	Trivy scans Docker image
6	Docker Build	Image creation
7	Image Push	Push to Docker Hub
8	GitOps Sync	Argo CD deploys to K8s
9	Monitoring	Prometheus & Grafana
10	Alerting	Alertmanager notifications

8. ADVANTAGES & DISADVANTAGES

Advantages	Disadvantages
1. Integrates security early in the CI/CD pipeline (DevSecOps)	1. Initial setup and configuration are complex
2. Automates build, test, security scanning, and deployment	2. Requires advanced knowledge of multiple tools
3. Ensures consistent deployments using GitOps (Argo CD)	3. Steep learning curve for beginners
4. Provides version-controlled and auditable deployments	4. Cloud infrastructure increases operational cost
5. Enables Infrastructure as Code using Terraform	5. Continuous tool maintenance is required
6. Reduces human errors through automation	6. Dependency on multiple third-party tools
7. Improves deployment reliability and stability	7. Troubleshooting across tools can be difficult
8. Supports easy rollback using Git-based configurations	8. Security scans increase CI/CD pipeline duration
9. Provides scalability and high availability via Kubernetes	9. Requires careful resource management

- **Security Tools and Their Purpose:**

Tool	Security Type	Purpose
SonarQube	SAST	Detect code issues & vulnerabilities
OWASP Dependency-Check	SCA	Scan third-party libraries
Trivy	Container Security	Detect image vulnerabilities
RBAC (K8s)	Access Control	Restrict cluster access
IAM (AWS)	Identity Management	Secure cloud access

9. CONCLUSION

The Secure GitOps-Based DevSecOps Platform implemented in this project demonstrates an effective approach to building, securing, deploying, and monitoring modern cloud-native applications. The project successfully integrates DevSecOps and GitOps principles to address the challenges of traditional software delivery pipelines, where security and deployment consistency are often treated as secondary concerns.

By automating the CI/CD pipeline using Jenkins and embedding security tools such as SonarQube, OWASP Dependency-Check, and Trivy, the platform ensures that security vulnerabilities are detected early in the software development lifecycle. This shift-left security approach significantly reduces the risk of insecure code and vulnerable container images being deployed into production environments.

The use of Docker and Kubernetes enables consistent application packaging, scalable deployments, and high availability through container orchestration features such as self-healing and auto-scaling. GitOps-based deployment using Argo CD ensures that all application and infrastructure changes are version-controlled, auditable, and easily reversible, improving deployment reliability and operational transparency.

Infrastructure automation using Terraform further enhances the system by providing consistent, repeatable, and scalable cloud infrastructure provisioning on AWS. Continuous monitoring and alerting using Prometheus, Grafana, and Alertmanager provide real-time visibility into application performance and infrastructure health, enabling proactive incident detection and faster resolution.

Overall, the project demonstrates how modern DevSecOps and GitOps practices can be effectively combined to create a secure, automated, and production-ready application delivery platform. The implementation not only fulfills academic objectives but also aligns with real-world industry practices, making it suitable for enterprise environments and professional DevSecOps adoption.

10. REFERENCES

1. Jenkins Documentation – *https://www.jenkins.io/documentation/*
2. SonarQube Documentation – *https://docs.sonarqube.org/*
3. Docker Documentation – *https://docs.docker.com/*
4. Kubernetes Documentation – *https://kubernetes.io/docs/*
5. Argo CD Documentation – *https://argo-cd.readthedocs.io/*
6. Terraform Documentation – *https://developer.hashicorp.com/terraform/docs*
7. OWASP Dependency-Check Documentation – *https://owasp.org/www-project-dependency-check/*
8. Trivy Vulnerability Scanner – *https://aquasecurity.github.io/trivy/*
9. Prometheus Monitoring System – *https://prometheus.io/docs/*
10. Grafana Documentation – *https://grafana.com/docs/*
11. Amazon Web Services (AWS) Documentation – *https://docs.aws.amazon.com/*
12. GitHub Documentation – *https://docs.github.com/*
13. DevSecOps Principles – OWASP Foundation – *https://owasp.org/*
14. GitOps Principles – CNCF – *https://www.cncf.io/*
15. Secure GitOps DevSecOps Platform (Reference Repository) – *https://github.com*